ARRAYS AS A DATA STRUCTURE

Real-world applications and a C++ Student Result Management System

Assignment topic	Arrays and their real-world applications
Selected application	Student Result Management System
Implementation language	C++

Prepared as an educational report. The accompanying C++ source code demonstrates the use of arrays in a practical application.

1. Introduction

A data structure is a way of organizing and storing data so a computer program can use it effectively. Arrays are among the basic data structures used in programming. They are especially useful when a program needs to handle many related values, such as students' marks, product prices, or daily temperature readings.

2. What is an array?

An array is a collection of elements stored under one variable name, with each element identified by an index. In C++, an ordinary fixed-size array stores elements of the same data type in consecutive memory locations. Indexing commonly starts at 0, so the first element is at index 0, the second at index 1, and so on.

Example in C++:

```
int marks[5] = {75, 40, 88, 52, 65};
cout << marks[0];  // prints 75
cout << marks[2];  // prints 88
```

Here, marks is an integer array with five elements. The program can use a loop to process each mark instead of creating five unrelated variables.

3. Why arrays are necessary

Reason	Explanation
Organization	Related values are grouped under one name, making them easier to manage.
Indexed access	A program can retrieve or update an element using its position.
Efficient processing	Loops can repeat the same operation across many elements.
Less repetitive code	One loop can handle many values rather than requiring separate statements for each value.
Calculations and analysis	Arrays make it straightforward to calculate totals, averages, maximums, minimums, and counts.

Arrays also have limitations. A traditional fixed-size array has a predetermined capacity, and inserting or deleting elements in the middle can require extra work. Larger real-world systems often combine arrays with dynamic containers, databases, and other data structures.

4. Real-world applications of arrays

4.1 Student result management systems

An array can store marks for students in a class. A loop can calculate the class average, find the highest and lowest marks, and count students who pass. Names and marks can be stored in parallel arrays using the same index for each student's record.

4.2 Banking and ATM systems

Arrays can temporarily hold a set of recent transaction amounts, daily totals, or data being processed in batches. Programs can loop through these values to calculate totals and detect unusual amounts. Production banking systems also rely on databases and secure transaction-processing systems.

4.3 Hospital management systems

Arrays can store a sequence of patient measurements, such as temperature readings taken over time, or lists of available bed identifiers. A program can examine the values to calculate averages or identify readings outside a defined range. Actual patient records require secure storage and appropriate clinical systems.

4.4 Inventory and stock management

Arrays can hold product prices, quantities, or stock codes. A program can calculate the total value of stock, update a quantity after a sale, and identify products whose stock is low. Larger inventories commonly use databases as well.

4.5 Computer games

Arrays can hold player scores, positions of game objects, or items in an inventory. Game loops can update and display many values efficiently. Larger games may use specialised collections and other data structures for more complex objects.

4.6 Weather and scientific data

An array can store hourly temperatures or repeated sensor readings. Programs can calculate a mean, find extremes, and plot the measurements in time order.

5. Selected application: Student Result Management System

The selected application is a console-based Student Result Management System written in C++. It demonstrates arrays in a way that is easy to understand and test. It stores student names and marks, displays results, calculates class statistics, and searches for a student.

Objectives

• Store up to 100 student names and marks during one run of the program.

• Display each student's mark and pass/fail status.

• Calculate total marks and the class average.

• Find the highest and lowest marks and identify the students who earned them.

• Count passes and failures and search for a student's name.

How arrays are used in the application

```
string studentNames[MAX_STUDENTS];
double studentMarks[MAX_STUDENTS];

studentNames[studentCount] = name;
studentMarks[studentCount] = mark;
++studentCount;
```

The program uses two parallel arrays. The name and mark at the same index belong to the same student. For example, studentNames[0] and studentMarks[0] represent the first student's name and mark. A loop visits the stored elements to display results and calculate the summary.

Main features

Feature	Purpose
Add student result	Accepts a student's name and a mark from 0 to 100.
Display all results	Lists the stored students and their pass/fail status.
Class summary	Calculates total, average, highest and lowest marks, and pass/fail counts.
Search student	Finds records by exact name.
Input validation	Rejects invalid menu choices and marks outside the allowed range.

6. Example results

Suppose five students obtain marks of 75, 40, 88, 52, and 65. The total is 320, the average is 64, the highest mark is 88, and the lowest mark is 40. If the pass mark is 50, four students pass and one student fails.

Student	Mark	Status
Student 1	75	Pass
Student 2	40	Fail
Student 3	88	Pass
Student 4	52	Pass
Student 5	65	Pass

7. Conclusion

Arrays are a fundamental data structure for storing and processing related values. They help programmers organize data, access values by index, and use loops to carry out repeated calculations. The Student Result Management System demonstrates these benefits by storing marks and producing useful class statistics. Although the example is small, the same principles apply to many practical applications. For a production system, persistent storage, access controls, privacy safeguards, and more scalable data structures would also be required.

8. How to run the C++ application

The source code is provided separately as main.cpp. Install a C++ compiler such as GCC/MinGW, open a terminal in the project folder, and compile with: g++ -std=c++11 -Wall -Wextra -pedantic main.cpp -o student_results. Run student_results.exe on Windows or ./student_results on Linux/macOS. Full instructions are included in README.md.